
CHAPITRE 2

Fonctions, collections et programmes fiables

Des fonctions que l'on peut expliquer, des données que l'on sait organiser

Le fil du chapitre

Une fois les boucles et les conditions en place, deux questions reviennent : **où ranger les données** et **comment réutiliser un calcul** ?

- Une fonction reçoit des valeurs et renvoie un résultat utilisable ailleurs.
- Une liste garde un ordre. Un tuple rassemble des positions fixes.
- Un dictionnaire associe une clé à une valeur. Un ensemble mémorise des éléments distincts.
- Une modification peut toucher plusieurs noms. Les tests rendent ces effets visibles.

Sur les diapositives « À vous », cherchez d'abord la réponse ou quelques lignes de code. La correction arrive sur la diapositive suivante.

Une fonction sépare le calcul de son utilisation

- `def` définit la fonction. Le corps s'exécute seulement lors d'un appel.
- Les paramètres nomment les valeurs reçues par cet appel.
- `return` termine l'appel et fournit son résultat. `print` affiche une valeur à l'écran.

Exemple de découpage. Le programme principal lit les données, appelle une fonction de calcul, puis affiche le résultat. On peut tester le calcul avec des valeurs écrites à la main, sans ouvrir de fichier ni demander une saisie.

Le **contrat** précise les entrées admises, le résultat attendu et les effets éventuels sur les objets reçus.

Un montant après remise

À vous. Écrivez `montant_net(montant, taux)`.

- `montant` est un nombre positif ou nul.
- `taux` est compris entre 0 et 1. Par exemple, 0,20 représente 20 %.
- La fonction renvoie le montant restant après la remise, sans l'afficher elle-même.

On suppose ici les entrées valides. Prévoyez le résultat de ces appels, puis écrivez la fonction.

```
# Appels à essayer après avoir écrit la fonction
print(montant_net(100, 0.20))
print(montant_net(50, 0))
print(montant_net(50, 1))
```

Quel test représente une remise nulle ? Une remise totale ?

Le montant restant : correction

```
def montant_net(montant, taux):  
    return montant * (1 - taux)  
  
print(montant_net(100, 0.20))  
print(montant_net(50, 0))  
print(montant_net(50, 1))
```

Résultat

```
80.0  
50  
0
```

1 - taux est la part conservée. Les cas 0 et 1 vérifient les deux bornes du contrat. L'appelant peut afficher le résultat ou l'utiliser dans un autre calcul.

Les noms à l'intérieur d'une fonction

À vous. Sans exécuter le programme, prévoyez les deux lignes affichées. Le nom `prix` apparaît dans la fonction et en dehors : désigne-t-il la même variable locale ?

```
prix = 100

def reduire(prix):
    prix = prix * 0.8
    return prix

print(reduire(prix))
print(prix)
```

La portée locale : correction

```
prix = 100

def reduire(prix):
    prix = prix * 0.8
    return prix

print(reduire(prix))
print(prix)
```

Résultat

```
80.0
100
```

Le paramètre `prix` est un nom local à l'appel. L'affectation dans la fonction change ce nom local. Le `prix` du programme principal conserve 100. Une fonction peut toutefois modifier une *liste* reçue en paramètre : nous allons distinguer ces deux situations.

Un tuple pour renvoyer deux résultats

Un **tuple** regroupe des valeurs dans un ordre fixé. La virgule le construit : (2, 5) contient deux valeurs, (2,) une seule. (2) est simplement le nombre 2.

À vous. Écrivez `min_max(valeurs)` pour une liste non vide. La fonction doit renvoyer d'abord le minimum, puis le maximum. Vous pouvez utiliser `min` et `max`.

```
valeurs = [4, 9, 2, 7]
# Écrire min_max, puis récupérer les deux résultats
# dans les variables bas et haut.
```

Que signifie l'ordre des deux valeurs renvoyées ?

Le retour multiple : correction

```
def min_max(valeurs):  
    return min(valeurs), max(valeurs)  
  
bas, haut = min_max([4, 9, 2, 7])  
print(bas, haut)
```

Résultat

```
2 9
```

`return min(...), max(...)` renvoie un tuple de deux éléments. L'affectation `bas, haut = ...` le déballe dans le même ordre.

Le nombre de noms doit correspondre au nombre de valeurs. On peut aussi écrire `premier, *milieu, dernier = [10, 20, 30, 40]` : `milieu` reçoit la liste `[20, 30]`.

Chaîne, liste et tuple

Type	Ce qu'il conserve	Modification
<code>str</code>	Du texte, caractère par caractère	On construit une nouvelle chaîne.
<code>list</code>	Des éléments dans un ordre donné	On peut ajouter, retirer ou remplacer un élément.
<code>tuple</code>	Des valeurs à des positions fixes	On ne peut pas remplacer une position.

Ces séquences ont une longueur, des indices et des tranches. Un tuple peut contenir une liste : ses positions restent fixes, mais cette liste peut encore changer.

Lire une liste

À vous. Prévoyez les quatre résultats. Quelle différence faites-vous entre un indice et une tranche ?

```
scores = [12, 18, 15]
print(scores[0])
print(scores[-1])
print(scores[1:3])
print(list("data"))
```

Que se passerait-il avec `scores[3]` ? Et avec `scores[1:10]` ?

Indices et tranches : correction

```
scores = [12, 18, 15]
print(scores[0])
print(scores[-1])
print(scores[1:3])
print(list("data"))
```

Résultat

```
12
15
[18, 15]
['d', 'a', 't', 'a']
```

L'indice 0 désigne le premier élément et -1 le dernier. La borne droite d'une tranche est exclue. `scores[3]` lève `IndexError`. La tranche `scores[1:10]` s'arrête à la fin et donne `[18, 15]`.

Ajouter un élément ou plusieurs

À vous. Exécutez ces instructions une par une. Avant chaque exécution, écrivez le contenu attendu de la liste et sa longueur.

```
a = [1, 2]
a.append([3, 4])

b = [1, 2]
b.extend([3, 4])
b.insert(0, 99)
```

`append` reçoit un objet. `extend` parcourt les éléments reçus. `insert` indique une position.

Les ajouts : correction

Instruction	Contenu après l'instruction	Longueur
<code>a.append([3, 4])</code>	<code>[1, 2, [3, 4]]</code>	3
<code>b.extend([3, 4])</code>	<code>[1, 2, 3, 4]</code>	4
<code>b.insert(0, 99)</code>	<code>[99, 1, 2, 3, 4]</code>	5

```
a = [1, 2]
a.append([3, 4])
b = [1, 2]
b.extend([3, 4])
b.insert(0, 99)
```

`append` ajoute la liste comme un seul élément. `extend` ajoute ses deux éléments séparément.

Retirer une valeur ou une position

À vous. Après chaque ligne, notez ce qu'il reste dans `valeurs`. Prévoyez aussi les valeurs de dernier et premier.

```
valeurs = [10, 20, 20, 30]
dernier = valeurs.pop()
premier = valeurs.pop(0)
valeurs.remove(20)
del valeurs[:1]
```

`pop` retire à une position et renvoie l'élément retiré. `remove` cherche une valeur et retire sa première occurrence.

Les suppressions : correction

Instruction	Liste restante	Valeur renvoyée
<code>pop()</code>	<code>[10, 20, 20]</code>	30
<code>pop(0)</code>	<code>[20, 20]</code>	10
<code>remove(20)</code>	<code>[20]</code>	None
<code>del valeurs[:1]</code>	<code>[]</code>	Instruction sans résultat

Un `pop` sur une position inexistante lève `IndexError`. Un `remove` sur une valeur absente lève `ValueError`.

`valeurs.index(x)` cherche la première position de `x` et échoue si `x` est absent.

`valeurs.count(x)` compte ses occurrences, sans modifier la liste.

Ce que renvoie append

À vous. Les deux affichages montrent-ils la même chose ?

```
scores = [12, 18]
resultat = scores.append(15)
print(resultat)
print(scores)
```

Pourquoi la ligne `scores = scores.append(15)` poserait-elle problème ?

Modifier un objet et renvoyer une valeur

```
scores = [12, 18]
resultat = scores.append(15)
print(resultat)
print(scores)
```

Résultat

```
None
[12, 18, 15]
```

`append` modifie la liste et renvoie `None`. Dans `scores = scores.append(15)`, l'ajout a lieu, puis le nom `scores` reçoit `None`.

`extend`, `insert`, `remove`, `sort` et `reverse` renvoient aussi `None`. `pop` est différent : il renvoie l'élément retiré.

Trier en conservant la liste de départ

À vous. Prévoyez les trois affichages.

```
notes = [12, 18, 15]
ordre = sorted(notes)
print(notes, ordre)
retour = notes.sort()
print(notes)
print(retour)
```

Quelle opération choisir si l'on veut garder les notes dans leur ordre initial ?

sort et sorted : correction

```
notes = [12, 18, 15]
ordre = sorted(notes)
print(notes, ordre)
retour = notes.sort()
print(notes)
print(retour)
```

Résultat

```
[12, 18, 15] [12, 15, 18]
[12, 15, 18]
None
```

`sorted` construit une nouvelle liste. `sort` trie la liste existante. L'option `reverse=True` demande un tri décroissant. La méthode `reverse()` inverse seulement l'ordre actuel, sans trier les valeurs.

Deux noms pour une même liste

À vous. Dessinez les listes et les noms qui les désignent, puis prévoyez les trois affichages.

```
a = [10, 20]
b = a
c = a.copy()
b.append(30)
c[0] = 99
print(a)
print(b)
print(c)
```

À quelle ligne crée-t-on une nouvelle liste ?

Alias et copie : correction

```
a = [10, 20]
b = a
c = a.copy()
b.append(30)
c[0] = 99
print(a)
print(b)
print(c)
```

Résultat

```
[10, 20, 30]
[10, 20, 30]
[99, 20]
```

`b = a` crée un autre nom pour la même liste. `a.copy()` crée une nouvelle liste. Un paramètre de fonction peut lui aussi désigner la liste de l'appelant : un `append` dans la fonction serait visible après l'appel.

Une copie peut encore partager des objets

À vous. Ici, chaque élément de la liste est lui-même une liste. Que vont afficher les deux `print` ?

```
original = [[1, 2], [3, 4]]
copie = original.copy()
copie[0].append(99)
print(original)
print(copie)
```

La copie concerne-t-elle la liste extérieure, les listes intérieures ou les deux ?

La copie superficielle : correction

```
original = [[1, 2], [3, 4]]  
copie = original.copy()  
copie[0].append(99)  
print(original)  
print(copie)
```

Résultat

```
[[1, 2, 99], [3, 4]]  
[[1, 2, 99], [3, 4]]
```

Les deux listes extérieures sont distinctes. Leurs éléments désignent encore les mêmes listes intérieures. `copy()` et la tranche `[:]` font toutes deux une copie superficielle. Ajouter un élément à `copie` seule ne l'ajouterait pas à `original`.

Un numéro avec chaque valeur

À vous. Affichez les noms avec une numérotation qui commence à 1. `enumerate(noms, start=1)` fournit à chaque tour un couple numéro-valeur.

```
noms = ["Ada", "Linus", "Guido"]  
# Écrire une boucle qui affiche le numéro et le nom.
```

Pourquoi n'a-t-on pas besoin d'incrémenter un compteur à la main ?

Le parcours avec enumerate : correction

```
noms = ["Ada", "Linus", "Guido"]  
for numero, nom in enumerate(noms, start=1):  
    print(numero, nom)
```

Résultat

```
1 Ada  
2 Linus  
3 Guido
```

enumerate avance le compteur en même temps que le parcours. Le déballage attribue les deux valeurs à `numero` et `nom`. Avec `start=1`, on obtient des numéros de présentation, pas les indices Python de la liste.

Associer des noms et des notes

À vous. `zip(noms, notes)` associe les éléments situés aux mêmes positions. Écrivez une boucle qui affiche chaque nom avec sa note. Prévoyez ce qui arrivera à Guido.

```
noms = ["Ada", "Linus", "Guido"]
notes = [18, 15]
# Parcourir zip(noms, notes) et afficher chaque couple.
```

zip s'arrête au plus court : correction

```
noms = ["Ada", "Linus", "Guido"]
notes = [18, 15]
for nom, note in zip(noms, notes):
    print(nom, note)
```

Résultat

```
Ada 18
Linus 15
```

Guido n'a pas de note correspondante. Le parcours s'arrête après deux couples. Si les longueurs doivent être égales, `zip(noms, notes, strict=True)` lève `ValueError` lorsque l'écart apparaît pendant le parcours (Python 3.10 ou plus récent).

Nettoyer une liste en gardant son ordre

À vous. Construisez une nouvelle liste qui retire les espaces aux extrémités de chaque nom et ignore les noms devenus vides. Gardez l'ordre d'arrivée.

```
noms = [" Ada ", "", " Linus", "   "]  
propres = []  
# Parcourir les noms, nettoyer, puis ajouter si non vide.
```

`texte.strip()` renvoie le texte sans espaces aux extrémités. Commencez par une boucle ordinaire. Que doit contenir `propres` après chaque tour ?

Le nettoyage par une boucle : correction

```
noms = [" Ada ", "", " Linus", "   "]
propres = []
for nom in noms:
    nettoye = nom.strip()
    if nettoye != "":
        propres.append(nettoye)
print(propres)
```

Résultat

```
['Ada', 'Linus']
```

Après chaque tour, propres contient les noms non vides déjà lus, nettoyés et dans le même ordre. On laisse la liste parcourue intacte. Supprimer ses éléments pendant le parcours risquerait de décaler les indices et de sauter un élément.

La même transformation en compréhension

Une compréhension construit une liste avec une expression de transformation, un parcours et éventuellement un filtre.

À vous. Réécrivez le nettoyage en une compréhension. Le test doit porter sur le nom *après* retrait des espaces.

```
noms = [" Ada ", "", " Linus", "   "]  
# propres = [expression for nom in noms if condition]
```

Le résultat doit rester celui de la boucle précédente. La liste de départ doit rester inchangée.

La compréhension : correction

```
noms = [" Ada ", "", " Linus", "  "]
propres = [nom.strip() for nom in noms if nom.strip() != ""]
print(propres)
```

Résultat

```
['Ada', 'Linus']
```

- `nom.strip()` est la valeur ajoutée à la nouvelle liste.
- `for nom in noms` définit le parcours.
- `if nom.strip() != ""` sélectionne les valeurs à garder.

La boucle précédente évite de calculer `strip()` deux fois et se complète plus facilement si plusieurs règles s'ajoutent. Les deux écritures conviennent ici.

Un dictionnaire associe une clé à une valeur

Un dictionnaire permet de nommer les informations plutôt que de mémoriser leur position.

```
client = {"id": 17, "ville": "Lille"}
```

- "ville" est une clé. "Lille" est sa valeur.
- `client[cle]` lit la valeur associée à la clé.
- `client[cle] = valeur` ajoute la clé ou remplace sa valeur si elle existe déjà.
- Une clé n'apparaît qu'une fois. Plusieurs clés peuvent avoir la même valeur.

Un dictionnaire peut représenter un client, un compteur de mots ou un total par catégorie.

Une clé présente, une clé absente

À vous. Prévoyez les résultats, puis ajoutez "segment": "retail" au dictionnaire.

```
client = {"id": 17, "ville": "Lille"}  
print(client["ville"])  
print(client.get("pays", "FR"))  
# Ajouter le segment, puis afficher client.
```

get(cle, default) renvoie la valeur par défaut si la clé manque. Que ferait client["pays"] ?
Le get précédent crée-t-il cette clé ?

Lire et compléter le dictionnaire : correction

```
client = {"id": 17, "ville": "Lille"}  
print(client["ville"])  
print(client.get("pays", "FR"))  
client["segment"] = "retail"  
print(client)
```

Résultat

```
Lille  
FR  
{'id': 17, 'ville': 'Lille', 'segment': 'retail'}
```

`client["pays"]` lève `KeyError`. `get` fournit une valeur de remplacement sans ajouter la clé. On utilise les crochets quand la présence est requise, et `get` quand l'absence est prévue.

Parcourir un dictionnaire qui évolue

À vous. Prévoyez le contenu de `cles` et de `instantane` après l'ajout. Écrivez ensuite une boucle qui affiche chaque clé avec sa valeur.

```
client = {"id": 17, "ville": "Lille"}
cles = client.keys()
instantane = list(client.keys())
client["pays"] = "FR"
# Afficher list(cles), instantane, puis parcourir client.items().
```

`keys()` donne les clés, `values()` les valeurs et `items()` les couples clé-valeur.

Une vue suit les changements : correction

```
client = {"id": 17, "ville": "Lille"}
cles = client.keys()
instantane = list(client.keys())
client["pays"] = "FR"
print(list(cles))
print(instantane)
for cle, valeur in client.items():
    print(cle, valeur)
```

Résultat

```
['id', 'ville', 'pays']
['id', 'ville']
id 17
ville Lille
pays FR
```

keys, values et items donnent des vues liées au dictionnaire. La liste instantane garde les clés présentes au moment de sa création. L'itération suit l'ordre d'insertion des clés.

Compter les mots d'une phrase

À vous. Produisez un dictionnaire qui associe chaque mot à son nombre d'occurrences.

- Passez le texte en minuscules, puis séparez-le avec `split()`.
- Retirez `.,!?:` aux extrémités de chaque morceau avec `strip(".,!?:")`.
- Ignorez les morceaux devenus vides. Les autres caractères restent tels quels.

```
texte = "Chat chat, chien."  
comptes = {}  
# Pour chaque mot nettoyé, augmenter son compteur.
```

Comment initialiser le compteur d'un mot que l'on rencontre pour la première fois ?

Le dictionnaire de fréquences : correction

```
texte = "Chat chat, chien."  
comptes = {}  
for morceau in texte.lower().split():  
    mot = morceau.strip(".,!?:;")  
    if mot != "":  
        comptes[mot] = comptes.get(mot, 0) + 1  
print(comptes)
```

Résultat

```
{'chat': 2, 'chien': 1}
```

Les états successifs sont {'chat': 1}, puis {'chat': 2}, puis {'chat': 2, 'chien': 1}.
get(mot, 0) fournit 0 au premier passage. Après chaque tour, le dictionnaire compte les mots déjà traités.

Un ensemble pour mémoriser les valeurs déjà vues

Un **ensemble** contient des éléments distincts, sans ordre de parcours garanti. `set()` crée un ensemble vide. `x in vus` teste la présence et `vus.add(x)` ajoute un élément.

À vous. Gardez la première occurrence de chaque identifiant, dans l'ordre d'arrivée. Utilisez une liste de résultat et un ensemble des valeurs vues.

```
identifiants = ["A", "B", "A", "C", "B"]
vus = set()
uniques = []
# Ajouter à uniques seulement les identifiants encore inconnus.
```

Quel rôle joue chacune des deux collections ?

La déduplication stable : correction

```
identifiants = ["A", "B", "A", "C", "B"]
vus = set()
uniques = []
for identifiant in identifiants:
    if identifiant not in vus:
        uniques.append(identifiant)
        vus.add(identifiant)
print(uniques)
```

Résultat

```
['A', 'B', 'C']
```

La liste passe par ["A"], ["A", "B"], reste identique au troisième tour, puis devient ["A", "B", "C"]. Le dernier B ne change rien. La liste conserve l'ordre. L'ensemble répond à « déjà vu ? ».

Comparer deux ensembles

À vous. Quels identifiants appartiennent à chacun des quatre résultats ? L'ordre dans lequel vous les écrivez n'a pas d'importance.

```
actifs = {"A", "B", "C"}  
contactes = {"B", "D"}  
# actifs & contactes  
# actifs | contactes  
# actifs - contactes  
# actifs ^ contactes
```

& signifie « dans les deux », | « dans au moins un », - « dans le premier seulement » et ^ « dans un seul des deux ».

Les opérations sur les ensembles : correction

Expression	Éléments du résultat
<code>actifs & contacts</code>	<code>{"B"}</code>
<code>actifs contacts</code>	<code>{"A", "B", "C", "D"}</code>
<code>actifs - contacts</code>	<code>{"A", "C"}</code>
<code>actifs ^ contacts</code>	<code>{"A", "C", "D"}</code>

`add(x)` ajoute un élément et `update(elements)` en ajoute plusieurs. `remove(x)` lève

`KeyError` si l'élément manque. `discard(x)` tolère cette absence.

Un ensemble ne possède pas d'indice comme `vus[0]`. Pour un affichage reproductible de ces chaînes, on peut afficher `sorted(vus)`, qui construit une liste triée.

Quelles valeurs peuvent servir de clés ?

Les clés d'un dictionnaire et les éléments d'un ensemble doivent être **hachables**. Python doit pouvoir leur associer une valeur de hachage stable pendant leur vie.

À vous. Parmi ces objets, lesquels peuvent être des clés de dictionnaire ?

```
17  
"Lille"  
(48.8, 2.3)  
[48.8, 2.3]  
("point", [48.8, 2.3])
```

Le fait que le dernier objet soit un tuple suffit-il ?

Les clés hachables : correction

Objet	Clé possible ?	Pourquoi ?
17	Oui	Un entier est hachable.
"Lille"	Oui	Une chaîne est hachable.
(48.8, 2.3)	Oui	Tous les éléments du tuple sont hachables.
[48.8, 2.3]	Non	Une liste est mutable et non hachable.
("point", [48.8, 2.3])	Non	Le tuple contient une liste non hachable.

L'immutabilité des positions du tuple ne rend pas ses éléments immuables. Une clé non hachable provoque `TypeError`. Pour les types usuels de ce chapitre, les nombres, les chaînes et les tuples de valeurs hachables conviennent.

Un tri à partir d'un champ

À vous. Triez les lignes par délai croissant et gardez lignes dans son ordre initial. L'argument `key` reçoit une fonction qui calcule la valeur à comparer pour chaque ligne.

```
lignes = [{"ville": "Lyon", "delai": 12},  
          {"ville": "Paris", "delai": 5}]  
# Créer ordre avec sorted et une clé de tri.
```

`lambda ligne: ligne["delai"]` est une petite fonction sans nom. Elle reçoit une ligne et renvoie son délai. Quelle ville doit apparaître en premier ?

La clé de tri : correction

```
lignes = [{"ville": "Lyon", "delai": 12},  
          {"ville": "Paris", "delai": 5}]  
ordre = sorted(lignes, key=lambda ligne: ligne["delai"])  
print([ligne["ville"] for ligne in ordre])  
print([ligne["ville"] for ligne in lignes])
```

Résultat

```
['Paris', 'Lyon']  
['Lyon', 'Paris']
```

La fonction de clé fournit 12 pour Lyon et 5 pour Paris. `sorted` classe selon ces nombres. `lignes.sort(key=lambda ligne: ligne["ville"])` trierait la liste existante selon les noms de villes. Les dictionnaires à l'intérieur restent les mêmes objets : ce tri ne les copie pas en profondeur.

Une erreur fait partie du contrat

Une fonction doit distinguer un résultat valide d'une entrée qu'elle ne peut pas traiter.

- `raise ValueError("message")` signale une valeur non admise et interrompt le chemin en cours.
- `try` délimite les instructions dont on veut traiter une erreur.
- `except ValueError` traite cette catégorie précise d'erreur.
- Sans traitement, l'exception remonte à l'appelant.

Par exemple, `int("sept")` échoue pendant la conversion. `int("-2")` réussit, mais -2 reste invalide si le contrat exige une quantité strictement positive.

Il faut donc vérifier à la fois la conversion et le domaine de la valeur obtenue.

Lire une quantité strictement positive

À vous. Écrivez `parse_positive_int(texte)`.

- Convertir `texte` avec `int` et renvoyer l'entier s'il est strictement positif.
- Si la conversion échoue, lever `ValueError("entier attendu")`.
- Si l'entier est nul ou négatif, lever `ValueError("valeur positive attendue")`.

```
# Cas à essayer après avoir écrit la fonction  
# "7", "0", "-2", "sept"
```

Prévoyez pour chaque entrée une valeur de retour ou le message d'erreur. Les appels invalides se testent séparément.

La conversion contrôlée : correction

```
def parse_positive_int(texte):  
    try:  
        valeur = int(texte)  
    except ValueError as erreur:  
        raise ValueError("entier attendu") from erreur  
    if valeur <= 0:  
        raise ValueError("valeur positive attendue")  
    return valeur
```

Entrée	Résultat
"7"	Retourne 7
"0" ou "-2"	ValueError : valeur positive attendue
"sept"	ValueError : entier attendu

`from erreur` conserve le lien avec l'échec de conversion initial. On intercepte seulement l'erreur que l'on sait expliquer ici.

Des tests choisis à partir du contrat

À vous. Pour la fonction précédente, trouvez un cas normal, une frontière et des cas invalides. Le nombre 1 mérite-t-il un test ? Pourquoi ?

`assert condition` ne fait rien si la condition est vraie. Sinon, il lève `AssertionError`.

```
# Reprendre la fonction parse_positive_int.  
# Ajouter les assertions pour "7" et "1".  
# Tester aussi "0", "-2" et "sept".
```

Une entrée invalide doit provoquer `ValueError`. Un résultat renvoyé normalement serait un échec du test. Les assertions servent ici à vérifier le code, pas à remplacer les validations de la fonction.

Vérifier aussi les erreurs : correction

La fonction est celle de la correction précédente.

```
assert parse_positive_int("7") == 7
assert parse_positive_int("1") == 1
for texte in ["0", "-2", "sept"]:
    try:
        parse_positive_int(texte)
    except ValueError:
        print(texte, ": refus attendu")
    else:
        raise AssertionError("refus attendu")
```

Résultat

```
0 : refus attendu
-2 : refus attendu
sept : refus attendu
```

Le `else` du `try` s'exécute seulement si aucune exception n'a eu lieu. Il détecte ici un faux succès. Le cas 1 vérifie le plus petit entier admis.

Un petit lot de commandes

On veut signaler les identifiants répétés et calculer le montant par client. **Règle de cet exercice** : on signale les doublons mais on conserve leurs montants. Les commandes annulées participent au contrôle des identifiants, mais pas aux totaux.

```
lignes = [  
    {"id": "01", "client": "Ada", "montant": 10, "statut": "ok"},  
    {"id": "02", "client": "Linus", "montant": 20, "statut": "annule"},  
    {"id": "01", "client": "Ada", "montant": 5, "statut": "ok"},  
    {"id": "03", "client": "Linus", "montant": 7, "statut": "ok"},  
]
```

Les champs sont présents et les montants sont valides. Chaque ligne décrit une observation à traiter.

Commandes : repérer les identifiants répétés

À vous. Reprenez les quatre lignes précédentes.

- Gardez dans un ensemble les identifiants déjà rencontrés.
- Lorsqu'un identifiant revient, ajoutez-le à une liste doublons.
- Conservez l'ordre de détection. Un identifiant qui revient trois fois sera signalé deux fois.

```
vus = set()
doublons = []
# Parcourir lignes et examiner ligne["id"].
```

Avant de coder, suivez l'état de `vus` et `doublons` sur les quatre lignes.

Les doublons : correction

Les données sont celles du petit lot de commandes.

```
vus = set()
doublons = []
for ligne in lignes:
    identifiant = ligne["id"]
    if identifiant in vus:
        doublons.append(identifiant)
    else:
        vus.add(identifiant)
print(doublons)
```

Résultat

```
['01']
```

Après les deux premières lignes, `vus` contient O1 et O2. La troisième ligne retrouve O1 et l'ajoute à `doublons`. La quatrième ajoute O3 à `vus`. Le statut n'intervient pas dans ce contrôle.

Commandes : calculer le montant par client

À vous. À partir du même lot, construisez totaux.

- Ignorez les lignes dont le statut vaut "annule".
- Utilisez le nom du client comme clé.
- Additionnez les montants des lignes conservées, y compris celles dont l'identifiant revient.

```
totaux = {}  
# Parcourir lignes, filtrer le statut, puis cumuler le montant.
```

Calculez d'abord à la main le total d'Ada et celui de Linus.

Les totaux : correction

Les données sont celles du petit lot de commandes.

```
totaux = {}
for ligne in lignes:
    if ligne["statut"] != "annule":
        client = ligne["client"]
        totaux[client] = totaux.get(client, 0) + ligne["montant"]
print(totaux)
```

Résultat

```
{'Ada': 15, 'Linus': 7}
```

Ada reçoit 10 puis 5. La commande de 20 pour Linus est annulée. Seule sa commande de 7 contribue au total. Le dictionnaire garde, pour chaque client, la somme des montants admis déjà lus.

Les deux calculs dans une fonction

La boucle met à jour les trois collections, puis la fonction renvoie un tuple.

```
def consolider(lignes):
    totaux, vus, doublons = {}, set(), []
    for ligne in lignes:
        identifiant = ligne["id"]
        if identifiant in vus:
            doublons.append(identifiant)
        else:
            vus.add(identifiant)
        if ligne["statut"] != "annule":
            client = ligne["client"]
            totaux[client] = totaux.get(client, 0) + ligne["montant"]
    return totaux, doublons
```

`consolider(lignes)` donne `({'Ada': 15, 'Linus': 7}, ['01'])`. `consolider([])` donne `({}, [])`. Les doublons sont signalés, pas éliminés : les retirer changerait le contrat et les totaux.

Le choix d'une collection

À vous. Associez chaque besoin à une structure et à l'opération qui vous intéresse.

1. Conserver les dossiers dans leur ordre d'arrivée.
2. Répondre souvent à « cet identifiant est-il déjà présent ? ».
3. Retrouver le total d'un client à partir de son identifiant.
4. Renvoyer des coordonnées dont les positions ne changent pas.

Pourquoi la recherche d'une valeur dans une liste devient-elle plus coûteuse quand la liste grandit ?

Le sens des données guide le choix

Besoin	Structure	Opération utile
Ordre d'arrivée	<code>list</code>	<code>append</code> , parcours dans l'ordre
Présence d'un identifiant	<code>set</code>	<code>identifiant in vus</code>
Total d'un client	<code>dict</code>	<code>totaux[client]</code>
Coordonnées fixes	<code>tuple</code>	Déballage des positions

L'accès à un indice de liste est direct. Chercher une valeur peut exiger de lire toute la liste. La recherche dans un ensemble ou par clé de dictionnaire est rapide en moyenne grâce au hachage, mais ces collections occupent aussi de la mémoire. Le chapitre 3 permettra de quantifier ces coûts.

Application : un index par identifiant

À vous. Écrivez `indexer(clients)` qui renvoie un dictionnaire associant chaque identifiant à sa position dans la liste.

- Les identifiants sont des chaînes et le champ "id" est toujours présent.
- Une liste vide doit donner un dictionnaire vide.
- Un identifiant répété doit provoquer `ValueError`, sans écrasement silencieux.

```
clients = [{"id": "C2", "ville": "Lyon"},  
           {"id": "C1", "ville": "Paris"}]  
# Écrire indexer avec enumerate et un dictionnaire.
```

Quels tests permettent de vérifier les trois règles du contrat ?

L'index secondaire : correction

```
def indexer(clients):  
    index = {}  
    for position, client in enumerate(clients):  
        identifiant = client["id"]  
        if identifiant in index:  
            raise ValueError("identifiant dupliqué")  
        index[identifiant] = position  
    return index
```

Entrée	Résultat
[{"id": "C2"}, {"id": "C1"}]	{"C2": 0, "C1": 1}
[]	{}
[{"id": "C2"}, {"id": "C2"}]	ValueError

Sans le test de présence, la seconde affectation remplacerait la première position. L'index reste valable tant que l'ordre et les identifiants de la liste ne changent pas.

Quelques méthodes à retrouver facilement

Structure	Opération	Effet
Liste	<code>a[1:3] = [7, 8]</code>	Remplace une tranche. La longueur peut changer.
Liste	<code>a.copy()</code>	Nouvelle liste extérieure, éléments partagés.
Dictionnaire	<code>d.update(autre)</code>	Ajoute des clés et remplace les valeurs des clés déjà présentes.
Dictionnaire	<code>d.pop(cle, default)</code>	Retire et renvoie la valeur, ou renvoie le défaut si la clé manque.
Ensemble	<code>s.update(elements)</code>	Ajoute plusieurs éléments distincts.

Sur un dictionnaire, `cle in d` cherche parmi les clés. Pour chercher parmi les valeurs, il faut le préciser avec `valeur in d.values()`.

Ce qu'il faut pouvoir expliquer

- Une fonction a des entrées, un résultat et un contrat. Ses noms locaux ne remplacent pas ceux de l'appelant.
- Une liste reçue en paramètre peut être modifiée. Un alias voit cette modification. Une copie superficielle partage encore les objets intérieurs.
- Une liste conserve un ordre, un ensemble des éléments distincts, un dictionnaire des associations. Un tuple fixe des positions.
- Une boucle construit un état que l'on peut suivre après chaque tour. Une compréhension convient quand la transformation reste simple.
- Les tests vérifient les résultats ordinaires, les frontières et les erreurs annoncées par le contrat.

Source : Notes du cours, chapitre 2. Python : structures de données, fonctions intégrées, exceptions.